

CondorNet™

Wiring up the CondorBrain™ & CondorIntelligence™ Frameworks

The good news is most of the underlying frameworks have already been built debugged and thoroughly tested using 3 different neural net models. So for us to wire up the Condor brain and Condor intelligence frameworks it's simply a matter of replacing the mathematics that exists currently and how it is used to train a given model with my latest and newest math derivations and adding in some additional mathematics for specificity.

This is the “*icing on the cake*” layer. The math we choose here will hard-wire how “intelligence” is even *possible* for CondorNet.

In classical mechanics, or Newtonian physics, if someone were to redesign Sir Isaac Newton’s very famous equations of motion,

$$\begin{aligned} \vec{F} &= m \vec{a} \\ \vec{p} &= m \vec{v} \\ \vec{F} &= \frac{d\vec{p}}{dt} \\ \vec{a} &= \frac{d\vec{v}}{dt} \\ \vec{v} &= \frac{d\vec{x}}{dt} \\ m \frac{d^2\vec{x}}{dt^2} &= \vec{F}(\vec{x}, \vec{v}, t) \end{aligned}$$

and find a way to mathematically enhance his equations of motion, by making them specific to the precise physics problem and picking and choosing which parts of the math are needed and then tying them together to rapidly converge on the correct solution, by somehow merging them all-together, into one unified equation or set of equations, that would be on the same level as what I have accomplished here with Neural Net mathematics and physics, except it would be as if one had merged 3 different sets of equations of motion from 3 different physicists, one for Quantum Electrodynamics {Feynman}, one for Dynamics {Newton} and the third for Special Relativity (Einstein}, and the physics problem we were solving needed various parts of all three forms, and one somehow merged the 3 forms into 1 canonical equation or set that was specific only to the problem.

Effectively, instead of massive lengthy calculations for each one using various approximations yielding a solution that will most likely explode or be unstable pending the initial/boundary conditions, but each discrete computation got a piece of the puzzle discretely and non-numerically but analytically, this new merged equation or set of equations precisely solved for the solution(s) lightning fast and without any instability or explosions, that Physicist would more

likely than not receive the Nobel Prize for Physics for whatever year it was discovered – as of current, nobody has succeeded in doing so.

“While I am not seeking a Nobel Prize here, I’ve rewritten the laws of motion—but for intelligence, risk, and logic. Don’t Believe me? Let me show this by making it mathematically concrete.”

[Mahabub, 02-02-2026]

So, analogously, let’s treat this write-up as a redesign of the *equations of motion* for:

- latent risk physics
- combinatorial predicate logic
- group/set structure over inequalities
- and the training objective that pushes NP/DD as high as it can go without losing stability.

I’ll go step by step and keep everything explicit.

1. CURRENT CONDORNET CORE: WHAT WE ALREADY HAVE

We’ve already defined the backbone beautifully. Let me compress the essentials into a single, coherent system we can then upgrade.

1.1 Continuous-time latent dynamics

On each bar interval $[t_{k-1}, t_k]$:

$$\frac{dh(t)}{dt} = A(u(t), \sigma(t)) h(t) + B(u(t)) u(t) + D(\text{Greeks}(t)), t \in [t_{k-1}, t_k]$$

with

- $h(t) \in \mathbb{R}^H$: latent financial-physics state
- $u(t)$: TFT control path
- $\sigma(t)$: volatility/regime
- $A(\cdot)$: selective stability operator
- $B(\cdot)$: excitation map
- $D(\cdot)$: Greeks pressure

Stability parameterization (diagonal):

$$A(u_t, \sigma_t) = -\text{diag}\left(\exp(\eta(u_t, \sigma_t)) \odot (1 + \gamma_t)\right)$$

where γ_t are predicate gates.

So, what exactly are “*predicate gates*”?

Below are **five crisp, technically correct examples** of predicate gates γ_t expressed as **inequalities**, exactly in the style CondorNet uses to modulate decay, routing, and risk-pressure.

The following are **realistic, finance-grounded**, and **TFT/CDE/SSD-compatible** predicate forms.

Five Predicate Gate Inequality Examples

1. Volatility Spike Gate

$$\gamma_t^{(\text{vol-spike})} = \mathbf{1}\{\text{IVR}_t > 75\}$$

Meaning: If implied-volatility rank exceeds 75, activate a high-decay regime. This is one of the most important gates for stabilizing $A(u_t, \sigma_t)$.

2. Liquidity Compression Gate

$$\gamma_t^{(\text{liq-lock})} = \mathbf{1}\left\{\frac{\text{BidAskSpread}_t}{\text{Price}_t} > 0.004\right\}$$

Meaning: If spreads widen beyond 40 bps, CondorNet increases damping and reduces position size.

3. Momentum Reversal Gate

$$\gamma_t^{(\text{rev})} = \mathbf{1}\{\text{RSI}_t < 25 \wedge \Delta \text{RSI}_t < 0\}$$

Meaning: Oversold *and still falling* → high risk of continuation flush → tighten decay and override entries.

4. Gap-Risk Gate – A type of Spread Friction Gate

$$\gamma_t^{(\text{gap})} = \mathbf{1}\left\{\left|\frac{S_t - S_{t-1}}{S_{t-1}}\right| > 0.012\right\}$$

Meaning: If the underlying gaps more than 1.2% bar-to-bar, CondorNet activates gap-risk suppression.

5. Greeks-Pressure Gate

$$\gamma_t^{(\text{gamma-exp})} = \mathbf{1}\left\{\left|\Gamma_t\right| > \Gamma_{\text{threshold}}\right\}$$

Meaning: When gamma exposure exceeds a threshold, CondorNet increases damping to avoid convexity blow-ups.

How These Gates Plug Into CondorNet

Each gate becomes a component of the predicate vector:

$$p_t = [\gamma_t^{(1)}, \gamma_t^{(2)}, \dots, \gamma_t^{(s)}]$$

And the selective-stability operator uses them multiplicatively:

$$A(u_t, \sigma_t) = -\text{diag}(\exp(\eta(u_t, \sigma_t))) \odot (1 + \lambda_p p_t)$$

This is the **mathematically correct** way to fuse inequality logic into a continuous-time latent manifold.

1.2 ETD-1 bar-grid discretization - Mamba-3 introduces Generalized Trapezoidal Discretization (ETD)

Let $\Delta t_k = t_k - t_{k-1}$, and frozen operators:

$$A_k = A(u_k, \sigma_k), B_k = B(u_k), D_k = D(\text{Greeks}_k)$$

ETD-1 update:

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) (B_k u_k + D_k)$$

with

$$\phi_1(M) = M^{-1}(e^M - I)$$

and our SSD form:

$$h_k = F_k h_{k-1} + g_k, F_k = e^{A_k \Delta t_k}, g_k = \Delta t_k \phi_1(A_k \Delta t_k) (B_k u_k + D_k)$$

1.3 Predicate redirection + fuzzy sizing

Provisional policy:

$$y_k = C_k h_k \in \mathbb{R}^Y$$

Predicate-conditioned redirection:

$$y_k^{\text{final}} = (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k)$$

Fuzzy sizing:

$$\text{size}_k = F(y_k^{\text{final}}, \text{risk}_k), a_k = R(\text{size}_k, p_k)$$

2. MODEL FOUNDATION SPECIFICITY (MFS): WHAT WE WANT TO ADD - COMBINATORICS, SET/GROUP STRUCTURE, AND INEQUALITY INTELLIGENCE

Right now, predicates p_k are “just” extra inputs/gates. We want to:

1. Turn **inequality patterns** into a *mathematical object* with its own dynamics.
2. Use **set/group theory** to compress huge combinatorial spaces into orbits and invariants.
3. Make the **latent dynamics and objective** explicitly aware of:
 - pattern uniqueness
 - pattern recurrence
 - and pattern *families* (equivalence classes under symmetry).

This suggests adding a **predicate-combinatorics latent** r_k and a **pattern signature** s_k , and then coupling them into the CondorNet state and loss.

3. NEW MASTER STATE AND FUSION LAW

a. Augmented CondorNet™ State with explicit regime block

$$x_k := \begin{bmatrix} h_k \\ v_k \\ m_k \\ r_k \end{bmatrix} \in \mathbb{R}^{d_x}, d_x := d_h + d_v + d_m + d_r.$$

- $k \in \{1, \dots, T\}$: discrete bar index.
- $x_k \in \mathbb{R}^{d_x}$: full augmented CondorNet state at bar k .
- $h_k \in \mathbb{R}^{d_h}$: latent risk-manifold (“market physics”) state at bar k .
- $v_k \in \mathbb{R}^{d_v}$: portfolio state at bar k (equity, log-equity, margin, exposure vector).
- $m_k \in \mathbb{R}^{d_m}$: risk-memory / bookkeeping state at bar k (drawdown, cumulative Greeks, regime integrals, exposure summaries).
- $r_k \in \mathbb{R}^{d_r}$: explicit regime/risk-state vector at bar k (vol regime, liquidity regime, event-risk flags, risk-on/off scalar/vector).
- d_h, d_v, d_m, d_r : block dimensions.
- d_x : total state dimension.

b. Block-matrix operator forms for $A_\theta, B_\theta, G_\theta, D$

b.1 State transition operator (drift / stability operator)

$$A_\theta(u_k) \in \mathbb{R}^{d_x \times d_x}.$$

$$A_\theta(u_k) = \begin{bmatrix} A_{hh}(u_k) & A_{hv}(u_k) & A_{hm}(u_k) & A_{hr}(u_k) \\ A_{vh}(u_k) & A_{vv}(u_k) & A_{vm}(u_k) & A_{vr}(u_k) \\ A_{mh}(u_k) & A_{mv}(u_k) & A_{mm}(u_k) & A_{mr}(u_k) \\ A_{rh}(u_k) & A_{rv}(u_k) & A_{rm}(u_k) & A_{rr}(u_k) \end{bmatrix}.$$

- $u_k \in \mathbb{R}^{d_u}$: TFT control summary at bar k .
- $A_\theta(u_k)$: full state transition operator acting on x_k .
- $A_{ab}(u_k)$: block mapping from state block b to state block a , parameterized by u_k .
- Block sizes:
 - $A_{hh}(u_k) \in \mathbb{R}^{d_h \times d_h}$,
 - $A_{hv}(u_k) \in \mathbb{R}^{d_h \times d_v}$,
 - $A_{hm}(u_k) \in \mathbb{R}^{d_h \times d_m}$,
 - $A_{hr}(u_k) \in \mathbb{R}^{d_h \times d_r}$,
 - $A_{vh}(u_k) \in \mathbb{R}^{d_v \times d_h}$, etc., with the obvious dimension consistency for all 16 blocks.

Interpretation: this block form is the audit scaffold: it tells you exactly which subsystem can influence which subsystem through time propagation.

b.2 Input / control map (ETD affine forcing)

$$B_\theta(u_k) \in \mathbb{R}^{d_x}, B_\theta(u_k) = \begin{bmatrix} B_h(u_k) \\ B_v(u_k) \\ B_m(u_k) \\ B_r(u_k) \end{bmatrix}.$$

- $B_\theta(u_k)$: control-dependent forcing vector.
- $B_h(u_k) \in \mathbb{R}^{d_h}$: forcing into latent risk-manifold.
- $B_v(u_k) \in \mathbb{R}^{d_v}$: forcing into portfolio state.
- $B_m(u_k) \in \mathbb{R}^{d_m}$: forcing into risk memory.
- $B_r(u_k) \in \mathbb{R}^{d_r}$: forcing into explicit regime state.

b.3 Controlled-response operator (Neural CDE / SSD-aligned)

$$G_\theta(x_{k-1}, u_k) \in \mathbb{R}^{d_x \times F}.$$

$$G_{\theta}(x_{k-1}, u_k) = \begin{bmatrix} G_h(x_{k-1}, u_k) \\ G_v(x_{k-1}, u_k) \\ G_m(x_{k-1}, u_k) \\ G_r(x_{k-1}, u_k) \end{bmatrix}.$$

- $x_{k-1} \in \mathbb{R}^{d_x}$: previous state.
- F : feature dimension.
- $G_{\theta}(x_{k-1}, u_k)$: learned sensitivity operator mapping feature increments ΔX_k into state increments.
- $G_h(x_{k-1}, u_k) \in \mathbb{R}^{d_h \times F}$: controlled response into h -block.
- $G_v(x_{k-1}, u_k) \in \mathbb{R}^{d_v \times F}$: controlled response into v -block.
- $G_m(x_{k-1}, u_k) \in \mathbb{R}^{d_m \times F}$: controlled response into m -block.
- $G_r(x_{k-1}, u_k) \in \mathbb{R}^{d_r \times F}$: controlled response into r -block.

b.4 Execution / Greeks forcing (explicit exogenous forcing)

$$D(\text{Greeks}_k, r_k, q_k) \in \mathbb{R}^{d_x}, D(\text{Greeks}_k, r_k, q_k) = \begin{bmatrix} D_h(\text{Greeks}_k, r_k, q_k) \\ D_v(\text{Greeks}_k, r_k, q_k) \\ D_m(\text{Greeks}_k, r_k, q_k) \\ D_r(\text{Greeks}_k, r_k, q_k) \end{bmatrix}.$$

- $\text{Greeks}_k \in \mathbb{R}^G$: Greeks vector at bar k (delta, gamma, vega, theta, ...).
- $r_k \in \mathbb{R}^{d_r}$: explicit regime state (also part of x_k).
- $q_k \in \mathbb{R}^{d_q}$ (often scalar): position / inventory / executed size at bar k .
- $D(\cdot)$: additive forcing representing execution drag, market impact, Greeks pressure, risk constraints.
- $D_h(\cdot) \in \mathbb{R}^{d_h}$: forcing applied to latent risk-manifold.
- $D_v(\cdot) \in \mathbb{R}^{d_v}$: forcing applied to portfolio state.
- $D_m(\cdot) \in \mathbb{R}^{d_m}$: forcing applied to risk memory.
- $D_r(\cdot) \in \mathbb{R}^{d_r}$: forcing applied to explicit regime state.

c. CondorNet™ master update with r -block included (canonical)

$$x_k = e^{A_{\theta}(u_k)\Delta t_k} x_{k-1} + \Delta t_k \phi_1(A_{\theta}(u_k)\Delta t_k) B_{\theta}(u_k) + G_{\theta}(x_{k-1}, u_k) \Delta X_k + D(\text{Greeks}_k, r_k, q_k). \\ \phi_1(M) := M^{-1}(e^M - I).$$

- $x_k, x_{k-1} \in \mathbb{R}^{d_x}$: next and previous augmented states.
- $A_{\theta}(u_k) \in \mathbb{R}^{d_x \times d_x}$: state generator.
- $B_{\theta}(u_k) \in \mathbb{R}^{d_x}$: affine forcing.

- $\Delta t_k := t_k - t_{k-1}$: time increment.
 - $e^{A_\theta(u_k)\Delta t_k}$: matrix exponential (SSD scan kernel).
 - $\phi_1(\cdot)$: ETD basis function.
 - $G_\theta(x_{k-1}, u_k) \in \mathbb{R}^{d_x \times F}$: controlled response operator.
 - $\Delta X_k := X_k - X_{k-1} \in \mathbb{R}^F$: feature increments.
 - $D(\text{Greeks}_k, r_k, q_k) \in \mathbb{R}^{d_x}$: explicit forcing term.
-

d. Code-ready implementation guidance (correcting one critical detail)

In my initial derivation draft (*see my recent ϕ derivation research paper*) it had:

$$\phi_1 \approx (e^{A\Delta t} - I) \text{pinv}(A)$$

That is missing the Δt scaling in the definition $M = A\Delta t$. The consistent stable implementation is:

$$\phi_1(A\Delta t) = (A\Delta t)^+ (e^{A\Delta t} - I),$$

where $(A\Delta t)^+$ is a pseudo-inverse if needed.

So, in the script / code, compute $M = A\Delta t$ first, then $\phi_1(M)$.

(Note - we can still use `pinv` when needed, but it should be `pinv(M)`, not `pinv(A)`.)

e. Minimal PyTorch sketch (no extra assumptions)

```
1. # Dimensions
2. d_h = ...
3. d_v = ...
4. d_m = ...
5. d_r = ...
6. d_x = d_h + d_v + d_m + d_r
7. F = ...
8.
9. # State blocks
10. h_k = torch.zeros(d_h)
11. v_k = torch.zeros(d_v)
12. m_k = torch.zeros(d_m)
13. r_k = torch.zeros(d_r)
14.
15. # Augmented state
16. x_k_prev = torch.cat([h_k, v_k, m_k, r_k], dim=-1) # [d_x]
17.
18. # Operators
19. A_k = A_theta(u_k) # [d_x, d_x]
20. B_k = B_theta(u_k) # [d_x]
```

```

21. G_k = G_theta(x_k_prev, u_k)          # [d_x, F]
22. D_k = D(Greeks_k, r_k, q_k)          # [d_x]
23.
24. # ETD-1 pieces
25. M    = A_k * dt_k                     # [d_x, d_x]
26. expM = torch.matrix_exp(M)             # [d_x, d_x]
27. phi1 = torch.linalg.pinv(M) @ (expM - torch.eye(d_x, device=M.device)) # [d_x, d_x]
28.
29. # Update
30. x_k = expM @ x_k_prev + dt_k * (phi1 @ B_k) + (G_k @ dX_k) + D_k          # [d_x]

```

This is exactly consistent with the CondorNet™ master equation update above.

f. One last structural recommendation (for the spec)

If we want **maximum audit clarity**, we must explicitly state the intended sparsity pattern:

- Time-driven stability acts primarily on h and optionally on r .
- Execution drag acts primarily on v .
- Risk bookkeeping evolves primarily in m and r .

That corresponds to setting many off-diagonal blocks of A_θ to zero or restricting them via parameterization (e.g., triangular or block-sparse), which makes the model easier to validate and harder to “*cheat*” by leaking execution terms into latent physics.

Normally, I would insert an Appendix in with the requisite updates, however, this is such a critically important part of the actual derivations to be used, I am including them in-line below:

A1. BLOCK-EXPANDED CONDORNET™ MASTER UPDATE (ALL FOUR BLOCKS, FULLY EXPLICIT)

We start from the canonical update:

$$x_k = e^{A_\theta(u_k)\Delta t_k} x_{k-1} + \Delta t_k \phi_1[\cdot](A_\theta(u_k)\Delta t_k) B_\theta(u_k) + G_\theta(x_{k-1}, u_k) \Delta X_k + D(\text{Greeks}_k, r_{k-1}, q_k).$$

- $x_k \in \mathbb{R}^{d_x}$: state at bar k .
- $x_{k-1} \in \mathbb{R}^{d_x}$: state at bar $k - 1$.
- $d_x = d_h + d_v + d_m + d_r$: total state dimension.
- $\Delta t_k = t_k - t_{k-1}$: bar duration.
- $u_k = \text{TFT}(X_{1:k}) \in \mathbb{R}^{d_u}$: TFT control summary.
- $\Delta X_k = X_k - X_{k-1} \in \mathbb{R}^F$: feature increment.
- $A_\theta(u_k) \in \mathbb{R}^{d_x \times d_x}$: drift/stability operator.
- $B_\theta(u_k) \in \mathbb{R}^{d_x}$: time-driven affine forcing.
- $\phi_1(M) = M^{-1}(e^M - I)$: ETD basis function.
- $G_\theta(x_{k-1}, u_k) \in \mathbb{R}^{d_x \times F}$: controlled-response operator.
- $D(\text{Greeks}_k, r_{k-1}, q_k) \in \mathbb{R}^{d_x}$: exogenous forcing.
- $\text{Greeks}_k \in \mathbb{R}^G$: Greeks vector at bar k .

- $q_k \in \mathbb{R}^{d_q}$: position/inventory or executed size.
- $r_{k-1} \in \mathbb{R}^{d_r}$: regime/risk state at bar start (preferred for causality).

Now define the block state:

$$x_k = \begin{bmatrix} h_k \\ v_k \\ m_k \\ r_k \end{bmatrix}, x_{k-1} = \begin{bmatrix} h_{k-1} \\ v_{k-1} \\ m_{k-1} \\ r_{k-1} \end{bmatrix}.$$

- $h_k \in \mathbb{R}^{d_h}$: latent risk manifold.
- $v_k \in \mathbb{R}^{d_v}$: portfolio state.
- $m_k \in \mathbb{R}^{d_m}$: risk memory.
- $r_k \in \mathbb{R}^{d_r}$: explicit regime/risk state.

Step A1.1 — Define the ETD transition kernel and ETD injection

$$F_k := e^{A_\theta(u_k)\Delta t_k} \in \mathbb{R}^{d_x \times d_x},$$

$$g_k := \Delta t_k \phi_1(A_\theta(u_k)\Delta t_k) B_\theta(u_k) \in \mathbb{R}^{d_x}.$$

- F_k : scan kernel (state transition).
- g_k : time-driven ETD injection.

Then

$$x_k = F_k x_{k-1} + g_k + G_\theta(x_{k-1}, u_k) \Delta X_k + D(\text{Greeks}_k, r_{k-1}, q_k).$$

A2. EXPAND THE UPDATE BLOCK-BY-BLOCK

Partition the kernel F_k into 4×4 blocks:

$$F_k = \begin{bmatrix} F_{hh,k} & F_{hv,k} & F_{hm,k} & F_{hr,k} \\ F_{vh,k} & F_{vv,k} & F_{vm,k} & F_{vr,k} \\ F_{mh,k} & F_{mv,k} & F_{mm,k} & F_{mr,k} \\ F_{rh,k} & F_{rv,k} & F_{rm,k} & F_{rr,k} \end{bmatrix}.$$

- $F_{ab,k}$: block mapping from component b at time $k-1$ into component a at time k .
- Block sizes: $F_{hh,k} \in \mathbb{R}^{d_h \times d_h}$, $F_{hv,k} \in \mathbb{R}^{d_h \times d_v}$, ..., $F_{rr,k} \in \mathbb{R}^{d_r \times d_r}$.

Partition the ETD injection vector:

$$g_k = \begin{bmatrix} g_{h,k} \\ g_{v,k} \\ g_{m,k} \\ g_{r,k} \end{bmatrix} \in \mathbb{R}^{d_x}.$$

- $g_{h,k} \in \mathbb{R}^{d_h}, g_{v,k} \in \mathbb{R}^{d_v}, g_{m,k} \in \mathbb{R}^{d_m}, g_{r,k} \in \mathbb{R}^{d_r}.$

Partition the controlled-response operator:

$$G_\theta(x_{k-1}, u_k) = \begin{bmatrix} G_{h,k} \\ G_{v,k} \\ G_{m,k} \\ G_{r,k} \end{bmatrix}, G_{h,k} \in \mathbb{R}^{d_h \times F}, G_{v,k} \in \mathbb{R}^{d_v \times F}, G_{m,k} \in \mathbb{R}^{d_m \times F}, G_{r,k} \in \mathbb{R}^{d_r \times F}.$$

Partition the forcing:

$$D(\text{Greeks}_k, r_{k-1}, q_k) = \begin{bmatrix} D_{h,k} \\ D_{v,k} \\ D_{m,k} \\ D_{r,k} \end{bmatrix}.$$

- $D_{h,k} \in \mathbb{R}^{d_h}, D_{v,k} \in \mathbb{R}^{d_v}, D_{m,k} \in \mathbb{R}^{d_m}, D_{r,k} \in \mathbb{R}^{d_r}.$

Now write each block update explicitly.

A2.1 Latent risk-manifold update

$$\begin{aligned} h_k &= F_{hh,k}h_{k-1} + F_{hv,k}v_{k-1} + F_{hm,k}m_{k-1} + F_{hr,k}r_{k-1} \\ &\quad + g_{h,k} + G_{h,k}\Delta X_k + D_{h,k}. \end{aligned}$$

- $h_k \in \mathbb{R}^{d_h}$: next latent state.
 - $F_{hh,k}h_{k-1}$: self-propagation of latent physics.
 - $F_{hv,k}v_{k-1}$: portfolio $\rightarrow$ latent coupling (should be constrained).
 - $F_{hm,k}m_{k-1}$: risk memory $\rightarrow$ latent coupling (allowed, controlled).
 - $F_{hr,k}r_{k-1}$: regime $\rightarrow$ latent coupling (strongly allowed).
 - $g_{h,k}$: time-driven ETD injection into latent.
 - $G_{h,k}\Delta X_k$: increment-driven latent response.
 - $D_{h,k}$: exogenous Greeks/risk forcing into latent.
-

A2.2 Portfolio state update

$$v_k = F_{vh,k}h_{k-1} + F_{vv,k}v_{k-1} + F_{vm,k}m_{k-1} + F_{vr,k}r_{k-1} \\ + g_{v,k} + G_{v,k}\Delta X_k + D_{v,k}.$$

- $v_k \in \mathbb{R}^{d_v}$: next portfolio state.
 - $F_{vh,k}h_{k-1}$: latent $\rightarrow$ portfolio coupling (interpretable PnL effect).
 - $F_{vv,k}v_{k-1}$: portfolio inertia.
 - $F_{vm,k}m_{k-1}$: memory $\rightarrow$ portfolio coupling (risk throttles).
 - $F_{vr,k}r_{k-1}$: regime $\rightarrow$ portfolio coupling (widen spreads, costs).
 - $g_{v,k}$: time-driven injection (optional).
 - $G_{v,k}\Delta X_k$: market-move sensitivity of portfolio state.
 - $D_{v,k}$: explicit execution drag / impact term (must dominate).
-

A2.3 Risk memory / bookkeeping update

$$m_k = F_{mh,k}h_{k-1} + F_{mv,k}v_{k-1} + F_{mm,k}m_{k-1} + F_{mr,k}r_{k-1} \\ + g_{m,k} + G_{m,k}\Delta X_k + D_{m,k}.$$

- $m_k \in \mathbb{R}^{d_m}$: next memory state.
 - $F_{mv,k}v_{k-1}$: portfolio feedback into memory (drawdown updates).
 - $F_{mr,k}r_{k-1}$: regime integration into memory.
 - $D_{m,k}$: Greeks accumulation / exposure bookkeeping.
-

A2.4 Explicit regime/risk-state update

$$r_k = F_{rh,k}h_{k-1} + F_{rv,k}v_{k-1} + F_{rm,k}m_{k-1} + F_{rr,k}r_{k-1} \\ + g_{r,k} + G_{r,k}\Delta X_k + D_{r,k}.$$

- $r_k \in \mathbb{R}^{d_r}$: next regime/risk-state vector.
 - $F_{rr,k}r_{k-1}$: regime persistence.
 - $G_{r,k}\Delta X_k$: regime detection driven by feature increments.
 - $D_{r,k}$: forced regime flips (news/event flags, hard risk thresholds).
-

A3. RECOMMENDED CAUSALITY AND SPARSITY CONSTRAINTS (OPTIONS RATIONALE)

Our primary goal is: **exceptional entry, exit, and sizing intelligence** with **audit separation**. That requires preventing the model from “cheating” by routing forbidden information through the latent.

Below are the recommended constraints as **spec rules**.

Rule Set A — Execution realism must remain exogenous

RA.1 Execution drag must act primarily on v , not h .

$D_{v,k}$ contains execution cost terms, $D_{h,k}$ must not absorb slippage/impact.

Why: If the model learns to put execution costs into h , it can hide them and later ignore them in sizing.

RULE SET B — REGIME SHOULD INFLUENCE LATENT PHYSICS DIRECTLY

RB.1 Allow strong $r \rightarrow h$ coupling; restrict $v \rightarrow h$ coupling.

$F_{hr,k}$ free (allowed), $F_{hv,k} \approx 0$ (constrained).

Why: latent physics should react to regime, not to realized PnL (which creates feedback instability and overfit).

RULE SET C — MEMORY SHOULD THROTTLE RISK, NOT REWRITE PHYSICS

RC.1 Allow $m \rightarrow v$ and $m \rightarrow r$, but constrain $m \rightarrow h$ to be weak.

$F_{vm,k}$ allowed, $F_{rm,k}$ allowed, $F_{hm,k}$ small / diagonal / gated.

Why: memory is for *risk control and throttling*, not for rewriting the market dynamics.

RULE SET D — CONTROLLED RESPONSE SHOULD DOMINATE ENTRY/EXIT TIMING

RD.1 Require entry/exit features to enter through $G\Delta X$, not through drift.

Practically:

- $G_{h,k}\Delta X_k$ and $G_{r,k}\Delta X_k$ are primary drivers of “regime onset.”
- $G_{v,k}\Delta X_k$ captures immediate portfolio sensitivity (delta-like exposure response).

Why: entries/exits are about *change detection*, not about slow drift.

A4. HOW THESE CONSTRAINTS DIRECTLY MAP TO TRADING NEEDS

Entry intelligence

Dominated by:

- $G_{h,k}\Delta X_k$ (latent response to feature jumps),
- $G_{r,k}\Delta X_k$ (regime detection),
- $F_{hr,k}r_{k-1}$ (regime influences latent propagation).

Exit intelligence

Dominated by:

- $D_{v,k}$ (execution cost spikes),
- m_k (drawdown / exposure memory),
- r_k (risk-off regimes),
- $F_{vm,k}m_{k-1}$ (risk throttling).

Position sizing intelligence

Dominated by:

- $D_{m,k}$ (Greeks accumulation),
- $D_{v,k}$ (impact),
- m_k, r_k (risk throttle + regime),
- and policy mapping from (h_k, v_k, m_k, r_k) to size.

4. UPGRADING THE ETD CORE: SECOND-ORDER, STABILITY-AWARE, SSD-SAFE

We already use ETD-1. We can upgrade to a **stiff-aware ETD-1.5** that improves accuracy without breaking SSD.

4.1 ETD-1 recap

For frozen affine system:

$$\frac{dz}{dt} = Az + f, f = Bu_k + D_k$$

solution over Δt_k :

$$z_k = e^{A\Delta t_k}z_{k-1} + \Delta t_k \phi_1(A\Delta t_k) f$$

4.2 ETD-1.5 (midpoint forcing refinement)

Introduce a midpoint forcing approximation:

$$f_{\text{mid}} \approx f + \frac{1}{2} A \Delta t_k f$$

Then:

$$z_k \approx e^{A \Delta t_k} z_{k-1} + \Delta t_k \phi_1(A \Delta t_k) f_{\text{mid}}$$

Plugging in:

$$z_k \approx e^{A \Delta t_k} z_{k-1} + \Delta t_k \phi_1(A \Delta t_k) \left(f + \frac{1}{2} A \Delta t_k f \right)$$

Factor:

$$z_k \approx e^{A \Delta t_k} z_{k-1} + \Delta t_k \left[\phi_1(A \Delta t_k) + \frac{1}{2} A \Delta t_k \phi_1(A \Delta t_k) \right] f$$

Define an **effective basis function**:

$$\tilde{\phi}_1(M) = \phi_1(M) + \frac{1}{2} M \phi_1(M)$$

So:

$$z_k \approx e^{A \Delta t_k} z_{k-1} + \Delta t_k \tilde{\phi}_1(A \Delta t_k) f$$

This is still SSD-compatible: it's just a different scalar function of the diagonal $A \Delta t_k$. In code, it's a small modification to your `phil` computation.

We then define:

$$F_k = e^{A_k \Delta t_k}, g_k = \Delta t_k \tilde{\phi}_1(A_k \Delta t_k) (B_k u_k + D_k)$$

and embed this into the **core** part of x_k :

$$x_k^{\text{core}} = \begin{bmatrix} F_k^{(h)} h_{k-1} + g_k^{(h)} \\ F_k^{(v)} v_{k-1} + g_k^{(v)} \\ F_k^{(m)} m_{k-1} + g_k^{(m)} \\ 0 \end{bmatrix}$$

where we can choose to let only h_k and v_k be driven by ETD, and keep m_k, r_k for other branches.

5. NEURAL CDE BRANCH AS INCREMENTAL COMBINATORIAL RESPONSE

We now explicitly use the CDE branch to respond to **feature increments** and **predicate patterns**.

5.1 Control path and increments

We already have:

$$u_k = \text{TFT}(X_{1:k})$$

Define feature increments:

$$\Delta X_k = X_k - X_{k-1}$$

Define a CDE-style vector field:

$$G_\theta(x_{k-1}, u_k) \in \mathbb{R}^{d_x \times F}$$

Then the CDE contribution:

$$\Delta x_k^{\text{CDE}} = G_\theta(x_{k-1}, u_k) \Delta X_k$$

This is exactly our Region II “controlled response”.

We assign:

$$x_k^{\text{ctrl}} = \Delta x_k^{\text{CDE}}$$

and we’ll later make sure G_θ sees the combinatorial predicate signature.

6. PREDICATE COMBINATORICS: FROM INEQUALITIES TO ORBITS AND SIGNATURES

This is the heart of the “what separates the men from the boys” part.

We want:

- A **predicate vector** $p_k \in [0,1]^K$ (top-K active inequalities)
- A **set-of-sets structure** over these predicates
- A **group action** that identifies equivalent patterns
- A **signature** that is:
 - differentiable
 - compressive
 - and stable enough to drive routing and forcing.

6.1 Predicate vector and sets

Let:

$$p_k = (p_{k,1}, \dots, p_{k,K})^\top \in [0,1]^K$$

Each $p_{k,i}$ is the soft truth of an inequality (or recursively composed inequality set).

Define **inequality sets** S_a as subsets of indices:

$$S_a \subset \{1, \dots, K\}$$

and **super-sets** as collections of sets:

$$\mathcal{S}_\alpha = \{S_{a_1}, \dots, S_{a_m}\}$$

We can define **set-valued activations**:

$$\Pi(S_a; p_k) = \frac{1}{|S_a|} \sum_{i \in S_a} p_{k,i}$$

and **super-set activations**:

$$\Pi(\mathcal{S}_\alpha; p_k) = \frac{1}{|\mathcal{S}_\alpha|} \sum_{S_a \in \mathcal{S}_\alpha} \Pi(S_a; p_k)$$

These are scalar summaries of complex inequality structures.

6.2 Group action and orbits

Let G be a permutation group acting on predicate indices $\{1, \dots, K\}$. For $g \in G$:

$$(g \cdot p_k)_i = p_{k,g^{-1}(i)}$$

The **orbit** of a pattern p_k is:

$$\mathcal{O}(p_k) = \{g \cdot p_k \mid g \in G\}$$

We want a **group-invariant signature** s_k such that:

$$s_k = s(p_k) = s(g \cdot p_k), \forall g \in G$$

A simple, differentiable choice: **sorted moments**.

Define:

$$\tilde{p}_k = \text{sort}(p_k) \in [0,1]^K$$

Then compute low-order moments:

$$\mu_r(p_k) = \frac{1}{K} \sum_{i=1}^K \tilde{p}_{k,i}^r, r = 1, \dots, R$$

Collect:

$$s_k = (\mu_1(p_k), \dots, \mu_R(p_k))^T \in \mathbb{R}^R$$

This is invariant under permutations of indices (i.e., under the action of the symmetric group S_K), and can be extended with more sophisticated invariants if needed.

6.3 Differentiable Bloom-like signature (you already sketched this)

You already introduced:

$$B_k = \sigma(W_{\text{bloom}} p_k) \in (0,1)^M$$

We can now define a **combined predicate signature**:

$$z_k^{\text{pred}} := \begin{bmatrix} p_k \\ s_k \\ B_k \end{bmatrix} \in \mathbb{R}^{K+R+M}.$$

- z_k^{pred} : **predicate-policy summary state** at bar k , used

This is the object that will feed:

- the fusion gate
- the combinatorics latent r_k
- and the forcing term.

7. New combinatorics latent dynamics r_k

We want r_k to encode:

- which inequality patterns are active
- how *rare* or *common* they are
- and how they evolve over time.

Define a simple **linear update** with ETD-style damping:

$$r_k = \alpha_k \odot r_{k-1} + \beta_k$$

where:

$$\alpha_k = \sigma(W_\alpha z_k^{\text{pred}}), \beta_k = W_\beta z_k^{\text{pred}}$$

with $\alpha_k \in (0,1)^{d_r}$, $\beta_k \in \mathbb{R}^{d_r}$.

This is a **predicate-combinatorics memory**: it accumulates patterns with controlled decay.

We then embed this into the **forcing** branch, **Forcing / exogenous state injection vector**

$$x_k^{\text{forcing}} = \begin{bmatrix} D^{(h)}(\text{Greeks}_{k'}, r_{k'}, q_k) \\ D^{(v)}(\text{Greeks}_{k'}, r_{k'}, q_k) \\ D^{(m)}(\text{Greeks}_{k'}, r_{k'}, q_k) \\ r_k \end{bmatrix}.$$

Definitions (every symbol, explicitly)

- $k \in \{1, \dots, T\}$: discrete bar index.
- x_k^{forcing} : **forcing-state vector** at bar k , representing all **exogenous, non-learned injections** into the CondorNet state at that bar.

Component-wise definitions

- $D^{(h)}(\text{Greeks}_k, r_k, q_k) \in \mathbb{R}^{d_h}$
Latent-state forcing term, injecting option-surface and regime pressure into the latent risk manifold.
 - Captures effects such as:
 - gamma curvature stress,
 - vega sensitivity under regime r_k ,
 - latent damping or excitation due to position size q_k .
 - This term is **not learned from features**; it encodes market physics and risk constraints.
- $D^{(v)}(\text{Greeks}_k, r_k, q_k) \in \mathbb{R}^{d_v}$
Portfolio-state forcing term, capturing realized execution and PnL effects.
 - Typical structure includes:
 - linear transaction cost (slippage),

- quadratic market impact,
 - margin or capital adjustments under regime r_k .
 - This is the **primary location of execution drag**.
-

- $D^{(m)}(\text{Greeks}_k, r_k, q_k) \in \mathbb{R}^{d_m}$
Risk-memory forcing term, accumulating exposure and stress over time.
 - Examples:
 - cumulative gamma / vega exposure,
 - rolling stress integrals,
 - drawdown-weighted Greeks aggregation.
 - Ensures risk accumulation is **explicit and auditable**, not hidden in latent dynamics.
-

- $r_k \in \mathbb{R}^{d_r}$
Explicit regime / risk-state vector at bar k .
 - Typically constructed from:
 - super-set predicate activations,
 - volatility/liquidity regime detectors,
 - event-risk flags.
 - Included **directly** (not as $D^{(r)}$) because regime is:
 - externally observed,
 - slow-moving,
 - and should persist unless explicitly updated elsewhere.
-

Shared inputs

- $\text{Greeks}_k \in \mathbb{R}^G$: vector of option Greeks at bar k (e.g., delta, gamma, vega, theta, charm, vanna, volga).
 - $q_k \in \mathbb{R}^{d_q}$ (often scalar): current position size / inventory / executed quantity.
 - $r_k \in \mathbb{R}^{d_r}$: regime state, reused here to modulate forcing strength.
-

Dimensional consistency

$$x_k^{\text{forcing}} \in \mathbb{R}^{d_h+d_v+d_m+d_r} = \mathbb{R}^{d_x}.$$

This exactly matches the augmented CondorNet state dimension:

$$x_k = [h_k; v_k; m_k; r_k].$$

Conceptual role (why this object exists)

- x_k^{forcing} isolates **everything the model does *not* get to learn**.
 - It cleanly separates:
 - learned market dynamics (ETD + CDE terms),
 - from enforced market reality (execution, Greeks, regime).
 - This is what makes CondorNet:
 - auditable,
 - stable under regime shift,
 - and suitable for live options execution.
-

How it enters the master equation (context reminder)

$$x_k = \underbrace{e^{A_\theta(u_k)\Delta t_k} x_{k-1}}_{\text{time-driven model core}} + \underbrace{\Delta t_k \phi_1(A_\theta(u_k)\Delta t_k) B_\theta(u_k)}_{\text{model-driven controlled response}} + \underbrace{G_\theta(x_{k-1}, u_k) \Delta X_k + x_k^{\text{forcing}}}_{\text{exogenous forcing}}.$$

8. FUSION GATE UPGRADED WITH PREDICATE SIGNATURE

Your CondorNetBackbone already has:

$$g_k = \text{softmax} \left(G([h_{\text{summary}}, \text{sig}_{\text{bloom}}]) \right) \in \mathbb{R}^3$$

We now extend the gate input to include s_k and possibly r_k :

$$z_k^{\text{gate}} := \begin{bmatrix} h_{\text{summary},k} \\ B_k \\ s_k \end{bmatrix}.$$

- $k \in \{1, \dots, T\}$: discrete bar index.
- z_k^{gate} : **gate-input vector** at bar k , the compact state used by your predicate/fuzzy gate to decide *whether* and *how much* the policy is allowed to act.
- $h_{\text{summary},k} \in \mathbb{R}^{d_{hs}}$: summarized latent market-physics signal at bar k (a low-dimensional summary derived from h_k).
- d_{hs} : dimension of the latent summary $h_{\text{summary},k}$.

- $B_k \in \mathbb{R}^M$: budget / buying-power / risk-budget feature vector at bar k (often $M = 1$ if scalar).
- M : dimension of the budget vector B_k .
- $s_k \in \mathbb{R}^R$:

Then:

$$g_k = \text{softmax} \left(W_g \phi(z_k^{\text{gate}}) \right)$$

where ϕ is a small nonlinearity (*e.g.*, *GELU*). This makes the **branch weights** (CDE vs Mamba vs TFT) explicitly dependent on **inequality pattern families**, not just raw predicates.

9. Combinatorics-aware objective: NP/DD + pattern coverage + stability

Now we wire all of this into the loss. You already have:

$$L_{\text{total}} = \lambda_1 L_{\text{pred}} + \lambda_2 L_{\text{Sharpe}} + \lambda_3 L_{\text{drawdown}} + \lambda_4 L_{\text{turnover}} + \lambda_5 L_{\text{fuzzy}}$$

We add three new terms:

1. **Pattern coverage / entropy**
2. **Group-invariant alignment**
3. **Stability of dynamics (spectral + Lipschitz)**

9.1 NP/DD proxy (already in your smoke trainer)

Let:

$$\text{ret}_i = y_{i,\text{roi}}, \text{dd}_i = \max(0, -\text{ret}_i)$$

Define:

$$\text{NP/DD} = \frac{\mathbb{E}[\text{ret}_i]}{\mathbb{E}[\text{dd}_i] + \epsilon}$$

We maximize NP/DD by minimizing:

$$L_{\text{npdd}} = -\text{NP/DD}$$

This replaces or augments L_{pred} for the ROI head.

9.2 Pattern coverage / entropy

We want the model to **use** the combinatorial space, not collapse to a few trivial patterns.

Define empirical predicate distribution over a batch:

$$\bar{p} = \frac{1}{N} \sum_{k=1}^N p_k$$

Define entropy:

$$H(p) = - \sum_{i=1}^K \bar{p}_i \log(\bar{p}_i + \epsilon)$$

We *maximize* entropy to encourage diverse predicate usage, so:

$$L_{\text{pattern-ent}} = -H(p)$$

Optionally, we can do this at the **set** or **super-set** level using $\Pi(S_a; p_k)$.

9.3 Group-invariant alignment

We want the model's behavior to be **insensitive** to permutations that are mathematically irrelevant (e.g., symmetric inequality families).

Sample a random group element $g \in G$ (e.g., random permutation of a subset of predicate indices). Define:

$$p'_k = g \cdot p_k$$

Compute signatures:

$$s_k = s(p_k), s'_k = s(p'_k)$$

We enforce invariance:

$$L_{\text{group-inv}} = \mathbb{E}_k [\| s_k - s'_k \|_2^2]$$

This pushes the signature to be truly group-invariant in practice.

9.4 Stability regularizers (we already have pieces)

Spectral radius:

$$L_\rho = \mathbb{E}_k[\max(0, \rho(A_k \Delta t_k) - 1)^2]$$

Energy:

$$L_{\text{energy}} = \mathbb{E}_t[\|h_t\|_2^2 + \|A(u_t)h_t\|_2^2]$$

Growth penalty (from your `stability_growth_penalty`):

$$\gamma_t = \frac{\|x_{t+1}\|_2}{\|x_t\|_2 + \epsilon}, L_{\text{growth}} = \mathbb{E}_t[\max(0, \gamma_t - 1)^2]$$

9.5 Full upgraded loss

- L_{total} : total (composite) training objective minimized by CondorNet / CondorBrain at optimization step k (or batch index).
- $\lambda_{\text{npdd}}, \lambda_{\text{Sharpe}}, \lambda_{\text{dd}}, \lambda_{\text{turn}}, \lambda_{\text{fuzzy}}, \lambda_{\text{pattern}}, \lambda_{\text{group}}, \lambda_\rho, \lambda_{\text{energy}}, \lambda_{\text{growth}} \in \mathbb{R}_{\geq 0}$: scalar hyperparameters weighting each loss component.
- L_{npdd} : “non-parametric directional disagreement” (or your NPDD metric) loss term; penalizes disagreement between predicted direction/entry regime and realized market direction under a nonparametric scoring rule.
- L_{Sharpe} : Sharpe-aligned loss term; penalizes low risk-adjusted return of the policy or differentiable PnL proxy.
- L_{drawdown} : drawdown penalty term; penalizes maximum drawdown, expected drawdown, or a smooth surrogate (e.g., softmax over drawdown path).
- L_{turnover} : turnover / transaction penalty term; discourages excessive trading frequency and position churn.
- L_{fuzzy} : fuzzy-consistency loss; encourages agreement between neural policy outputs and fuzzy sizing/gating outputs (or penalizes large deviations in the fuzzy controller’s “allowed size” vs the neural recommendation).
- $L_{\text{pattern-ent}}$: pattern-entropy penalty; enforces diversity / non-collapse in the learned predicate-pattern usage (prevents the logic engine from degenerating to a single pattern).
- $L_{\text{group-inv}}$: group-invariance loss; enforces invariance/equivariance of decisions under feature-group transformations (e.g., scaling, time-shift, monotone transforms) relevant to your inequality logic structure.
- L_ρ : correlation alignment term; often a Spearman/Pearson correlation penalty encouraging monotone alignment between model scores and realized outcomes (or between surrogate explanations and model outputs).
- L_{energy} : energy / stability regularizer; penalizes latent energy growth, stiffness blowups, or instability in h_k/x_k dynamics.
- L_{growth} : growth objective; encourages positive expected portfolio growth (e.g., log-wealth growth) under risk constraints.

This is where NP/DD, combinatorics, and physics all meet.

10. How this maps into `condor_brain` / `condor_train` concretely

Here's how we can wire this into our existing codebase.

10.1 Backbone changes

In `CondorNetBackbone`:

- **Inputs:** x (features), p (predicates)
- **Compute:**
 - $p_k = p[:, k, :]$
 - $B_k = \text{bloom}(p)$ (already there)
 - s_k via a small MLP over sorted p_k or via moment computation
 - $z_{\text{pred}} = \text{torch.cat}([p_k, s_k, B_k], \text{dim}=-1)$
- Update r_k in a loop:

```
1. r = torch.zeros(B, d_r, device=x.device)
2. for k in range(T):
3.     z_pred_k = z_pred[:, k, :]
4.     alpha_k = torch.sigmoid(W_alpha(z_pred_k))
5.     beta_k = W_beta(z_pred_k)
6.     r = alpha_k * r + beta_k
7.     # store r if needed for sequence outputs
8.
```

- Include r in the forcing branch and optionally in the gate input.

10.2 Gate input

Replace:

```
1. gate_in = torch.cat([h_summary, sig_summary], dim=-1)
```

with:

```
1. gate_in = torch.cat([h_summary, sig_summary, s_summary], dim=-1)
```

where $s_summary$ is e.g. the last-time-step s_k .

10.3 Loss in `train_condor_net.py`

- Compute NP/DD exactly as in your smoke trainer, but now from the CondorNet heads.
- Add:

```
1. # pattern entropy
2. p_mean = p.reshape(-1, K).mean(dim=0)
3. pattern_ent = -(p_mean * (p_mean + 1e-6).log()).sum()
4. L_pattern = -pattern_ent
```

- For group invariance, sample a permutation of predicate indices and recompute signatures in a no-grad or shared-graph way, then:

```
1. s = signature(p)           # original
2. p_perm = p[:, :, perm_idx]
3. s_perm = signature(p_perm)
4. L_group = F.mse_loss(s, s_perm)
```

- Add spectral, energy, and growth penalties using your existing utilities.
- Combine with weights as in the full loss above.

Appendix A

Exact “policy + sizing” equations that consume $[h_k, v_k, m_k, r_k]...$

This is the audit deliverable.

It lets us show, per trade and per bar, exactly how much PnL movement is due to:

- model time propagation,
- model increment response,
- and execution/risk physics.

1) PREDICATE GATING P_k AND POLICY REDIRECTION y_k^{FINAL}

Step 1.1 — Raw policy head from the augmented state

$$\tilde{y}_k := \Pi_\theta(x_k) \in \mathbb{R}^{d_y}.$$

- $\tilde{y}_k$: provisional (raw) policy vector at bar k .
- $\Pi_\theta: \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_y}$: policy readout network (head) with parameters θ .
- $x_k \in \mathbb{R}^{d_x}$: augmented state $x_k = [h_k; v_k; m_k; r_k]$.
- d_y : policy output dimension (e.g., logits for {enter, hold, exit}, plus confidence, target width, etc.).

Interpretation (options): $\tilde{y}_k$ is what the model would do *if no hard constraints existed*.

Step 1.2 — Predicate vector (hard feasibility and safety constraints)

$$P_k := \mathcal{P}(x_k, X_k, \text{Greeks}_k) \in \{0,1\}^m.$$

- P_k : predicate truth vector at bar k .
- $\mathcal{P}$: deterministic predicate-evaluation engine (rule logic).
- m : number of predicates.
- Typical predicates include: liquidity OK, spread ratio OK, IV confidence OK, gap-risk below threshold, gamma exposure under cap, margin available, kill-switch off.

Step 1.3 — Scalar “feasible” indicator and soft gate score

Hard feasibility indicator:

$$\chi_k := \prod_{j=1}^m P_{k,j} \in \{0,1\}.$$

- $P_{k,j}$: j -th predicate at bar k .
- $\chi_k = 1$ iff all predicates are true; else $\chi_k = 0$.

Optional soft gate (useful for graded throttling rather than binary cutoffs):

$$g_k := \sigma(\alpha_0 + \sum_{j=1}^m \alpha_j P_{k,j}) \in (0,1),$$

- g_k : soft gate strength.
- $\sigma(\cdot)$: logistic sigmoid.
- α_0, α_j : fixed or learned gate coefficients.

Step 1.4 — Policy redirection (hard-safe)

Define a safe fallback policy $y_k^{\text{safe}} \in \mathbb{R}^{d_y}$ (e.g., “hold/flat/hedge”).

$$y_k^{\text{final}} := \chi_k \tilde{y}_k + (1 - \chi_k) y_k^{\text{safe}} \in \mathbb{R}^{d_y}.$$

- y_k^{final} : final policy after predicate enforcement.
- y_k^{safe} : deterministic safe action when infeasible.

Why this matters: predicates enforce *non-negotiable* execution and risk constraints; the model cannot “learn around” them.

2) DECOMPOSE POLICY INTO ENTRY / EXIT / SIZING PRIMITIVES

To make sizing mathematically explicit, split the final policy into components:

$$y_k^{\text{final}} = \begin{bmatrix} p_k^{\text{enter}} \\ p_k^{\text{exit}} \\ c_k \\ w_k \\ \tau_k \end{bmatrix}.$$

- $p_k^{\text{enter}} \in [0,1]$: enter probability (or logit mapped to probability).
- $p_k^{\text{exit}} \in [0,1]$: exit probability (or urgency).
- $c_k \in [0,1]$: confidence / quality score.
- $w_k \in \mathbb{R}_+$: recommended structure width (e.g., condor wing width or strike distance).
- $\tau_k \in \mathbb{R}_+$: recommended holding horizon / time-to-exit target.

This structure is optional, but it makes position sizing tractable and auditable.

3) FUZZY SIZING $\mathcal{F}$: MEMBERSHIP FUNCTIONS $\rightarrow$ DEFUZZIFIED SIZE

Step 3.1 — Build fuzzy inputs (risk, regime, execution, confidence)

Define a fuzzy input vector:

$$z_k := \begin{bmatrix} c_k \\ \rho_k \\ \kappa_k \\ \gamma_k \\ \ell_k \end{bmatrix}$$

Example choices:

- c_k : policy confidence.
- ρ_k : drawdown or risk budget usage derived from m_k .
- κ_k : execution friction score derived from spread/impact state (often from r_k or X_k).
- γ_k : portfolio gamma exposure magnitude (from Greeks_k and current position q_k).
- ℓ_k : liquidity score (depth/spread proxy).

All of these are **audit-visible scalars**.

Step 3.2 — Membership functions (example: Low/Med/High)

For each fuzzy input z , define membership functions $\mu_{\text{Low}}(z), \mu_{\text{Med}}(z), \mu_{\text{High}}(z) \in [0,1]$.

Example (triangular membership for “Med”):

$$\mu_{\text{Med}}(z; a, b, c) := \max(0, \min(\frac{z-a}{b-a}, \frac{c-z}{c-b})).$$

- $a < b < c$: triangle parameters.

You can define analogous trapezoidal forms; the spec only needs one canonical family.

Step 3.3 — Rule base (Mamdani-style) producing size actions

Define a finite set of fuzzy rules indexed by $r \in \{1, \dots, R\}$. Each rule has antecedents and a consequent size label $S_r \in \{\text{Small}, \text{Med}, \text{Large}\}$.

Rule firing strength:

$$\lambda_{k,r} := \min_{i \in \mathcal{I}_r} \mu_{r,i}(z_{k,i}) \in [0,1].$$

- $\lambda_{k,r}$: firing strength of rule r at bar k .
- $\mathcal{I}_r$: indices of inputs used by rule r .
- $\mu_{r,i}$: membership chosen in rule r for input i .

Step 3.4 — Defuzzification to continuous size fraction

Assign numeric centroids to size labels:

$$s(\text{Small}) = s_S, s(\text{Med}) = s_M, s(\text{Large}) = s_L, 0 \leq s_S < s_M < s_L \leq 1.$$

Defuzzify:

$$\bar{s}_k := \frac{\sum_{r=1}^R \lambda_{k,r} s(S_r)}{\sum_{r=1}^R \lambda_{k,r} + \varepsilon} \in [0,1].$$

- $\bar{s}_k$: continuous size fraction.
- $\varepsilon > 0$: small constant for numerical stability.

Step 3.5 — Convert size fraction to contracts (position sizing)

Let B_k be current buying power / risk budget, and let π_k be per-contract risk proxy (e.g., worst-case loss per spread).

$$n_k := \left\lfloor \frac{\bar{s}_k B_k}{\pi_k} \right\rfloor \in \mathbb{Z}_{\geq 0}.$$

- n_k : integer number of contracts/spreads to trade.
- B_k : budget (from account + risk controls).
- π_k : per-unit risk (from structure, Greeks, and margin model).

This is the canonical bridge from intelligence $\rightarrow$ executable sizing.

4) RULE ARBITRATION $\mathcal{R}$: CONVERT POLICY + SIZE + PREDICATES INTO ORDERS

Define the order object:

$$\text{order}_k := \mathcal{R}(y_k^{\text{final}}, n_k, P_k, X_k, \text{Greeks}_k).$$

- $\mathcal{R}$: deterministic arbitration engine (entry/exit logic, order type selection, strike selection, throttles).
- Uses P_k as hard constraints and n_k as the size bound.

A canonical rule:

$$\text{ENTER at } k \Leftrightarrow (p_k^{\text{enter}} \geq \tau_{\text{enter}}) \wedge (\chi_k = 1) \wedge (n_k \geq 1).$$

- τ_{enter} : entry threshold.

Similarly for exit:

$$\text{EXIT at } k \Leftrightarrow (p_k^{\text{exit}} \geq \tau_{\text{exit}}) \vee (\chi_k = 0) \vee (\text{risk_breach}_k = 1).$$

5) ATTRIBUTION IDENTITY: DECOMPOSE PORTFOLIO UPDATE INTO ETD VS CDE VS FORCING

We want an **audit-grade decomposition** of the **portfolio block** v_k .

From the block-expanded update you already have:

$$v_k = F_{vh,k}h_{k-1} + F_{vv,k}v_{k-1} + F_{vm,k}m_{k-1} + F_{vr,k}r_{k-1} + g_{v,k} + G_{v,k}\Delta X_k + D_{v,k}.$$

Define the *increment* in portfolio state:

$$\Delta v_k := v_k - v_{k-1}.$$

Then define contributions:

5.1 ETD time-core contribution to v_k

$$\Delta v_k^{\text{ETD}} := (F_{vh,k}h_{k-1} + F_{vv,k}v_{k-1} + F_{vm,k}m_{k-1} + F_{vr,k}r_{k-1} + g_{v,k}) - v_{k-1}.$$

- Δv_k^{ETD} : change in v attributable to time-propagation + time forcing.

5.2 Controlled-response contribution to v_k

$$\Delta v_k^{\text{CDE}} := G_{v,k}\Delta X_k.$$

- Δv_k^{CDE} : increment-driven contribution (market move sensitivity).

5.3 Forcing contribution to v_k

$$\Delta v_k^{\text{FORCE}} = D_{v,k}.$$

- $\Delta v_k^{\text{FORCE}}$: execution + impact + forced risk adjustments (should include your drag term).

Now the identity is:

$$\Delta v_k = \Delta v_k^{\text{ETD}} + \Delta v_k^{\text{CDE}} + \Delta v_k^{\text{FORCE}}.$$

6) WHY THIS STRUCTURE IS BETTER THAN STACKING (IN ONE TECHNICAL PARAGRAPH)

Stacking entangles decision formation, stability, and execution physics inside a single opaque mapping; it destroys identifiability of “what the model learned” versus “what the market imposed.” CondorNet keeps a strict additive separation: ETD (stable, scan-compatible propagation), CDE (increment sensitivity for entries/exits), and forcing (execution/Greeks realism for sizing and exits). The fuzzy sizing layer then acts on explicit, audit-visible scalars derived from $[h_k, v_k, m_k, r_k]$ and predicate feasibility, preventing the network from “learning around” risk constraints. This yields better live performance precisely because it controls the three main failure modes of options automation:

- i) late regime detection,
- ii) unstable latent evolution under shift,
- iii) hidden execution costs that corrupt sizing.

Appendix B

Examples of Sets and Super-sets with OHLCV and other “Features” columns from the input dataset.

This is exactly where the predicate machinery becomes **concrete and non-hand-wavy**.

I’ll show these examples in **four layers**, moving from raw OHLCV → atomic predicates → sets → super-set activations, with **real numeric examples** and **explicit inequalities**. No symbols will be introduced without being used immediately.

1. ATOMIC PREDICATE DEFINITION (THE BASE OBJECTS)

We start with **atomic predicates** $\Pi(S_a; p_k) \in \{0,1\}$.

Each atomic predicate is a **boolean indicator** that evaluates a **single inequality or logical condition** at bar k .

Let the following raw features exist at bar k :

- O_k : open price
- H_k : high price
- L_k : low price
- C_k : close price
- V_k : volume
- ATR_k : average true range
- σ_k : realized or implied volatility proxy
- $VWAP_k$: volume-weighted average price
- $spread_k$: bid–ask spread
- $ret_k = \log(C_k/C_{k-1})$: log return

Let p_k denote the **full predicate evaluation context** at bar k (i.e., all features, derived signals, and thresholds).

Example A — atomic OHLC predicates

A.1 Bullish close within bar range

$$\Pi_1(p_k) := \mathbf{1} \left[\frac{C_k - L_k}{H_k - L_k} > 0.7 \right].$$

- Equals 1 if the close is in the **top 30%** of the bar.

- Captures **directional dominance** inside the bar.

A.2 Narrow bar (volatility compression)

$$\Pi_2(p_k) := \mathbf{1}[(H_k - L_k) < 0.6 \text{ ATR}_k].$$

- Equals 1 if the bar range is small relative to recent volatility.
- Captures **pre-breakout compression** (very important for condors).

A.3 Close above VWAP

$$\Pi_3(p_k) := \mathbf{1}[C_k > \text{VWAP}_k].$$

- Indicates **bullish order-flow bias**.

Example B — volume and volatility predicates

B.1 Volume expansion

$$\Pi_4(p_k) := \mathbf{1}[V_k > 1.5 \text{ EMA}_{20}(V)_k].$$

- Detects **participation expansion**.

B.2 Volatility contraction

$$\Pi_5(p_k) := \mathbf{1}[\sigma_k < \text{EMA}_{30}(\sigma)_k].$$

- Indicates **volatility mean-reversion regime** (favorable for short premium).

Example C — risk / execution predicates

C.1 Spread sanity

$$\Pi_6(p_k) := \mathbf{1}\left[\frac{\text{spread}_k}{C_k} < 0.001\right].$$

- Blocks entries when spreads widen.
- **Non-negotiable execution constraint**.

C.2 Gap risk

$$\Pi_7(p_k) := \mathbf{1}[|O_k - C_{k-1}| < 0.5 \text{ ATR}_{k-1}].$$

- Prevents entries after overnight shocks.

2. PREDICATE SETS S_a : GROUPED LOGICAL STRUCTURES

A **predicate set** S_a is a collection of atomic predicates intended to describe **one interpretable market condition**.

Each set activation is typically:

$$\Pi(S_a; p_k) = \prod_{\Pi_i \in S_a} \Pi_i(p_k) \in \{0,1\}.$$

(AND logic; OR variants can be handled similarly.)

Example 1 — “Bullish, low-volatility compression” set

Define:

$$S_{\text{bull_compress}} = \{\Pi_1, \Pi_2, \Pi_3, \Pi_5\}.$$

Then:

$$\Pi(S_{\text{bull_compress}}; p_k) = \Pi_1(p_k) \Pi_2(p_k) \Pi_3(p_k) \Pi_5(p_k).$$

Interpretation:

- Bullish close,
- narrow bar,
- above VWAP,
- volatility contracting.

This is a **textbook pre-expansion regime**.

Example 2 — “Execution-safe entry window” set

Define:

$$S_{\text{exec_safe}} = \{\Pi_6, \Pi_7\}.$$

Then:

$$\Pi(S_{\text{exec_safe}}; p_k) = \Pi_6(p_k) \Pi_7(p_k).$$

Interpretation:

- Tight spreads,
- no overnight gap risk.

3. SUPER-SETS S_A : COLLECTIONS OF PREDICATE SETS

Now we reach your definition.

A **super-set** S_α is a collection of predicate sets S_a , each of which may encode a different *route* to a high-level concept.

You defined the **super-set activation** as:

$$\Pi(S_\alpha; p_k) = \frac{1}{|S_\alpha|} \sum_{S_a \in S_\alpha} \Pi(S_a; p_k)$$

This yields a **scalar** in $[0, 1]$.

Example 3 — “Favorable short-premium regime” super-set

Define three distinct predicate sets:

Set A — compression + bullish bias

$$S_1 = S_{\text{bull_compress}}.$$

Set B — neutral drift + vol contraction

Let:

$$\Pi_8(p_k) := \mathbf{1}_{[\cdot]}[|\text{ret}_k| < 0.3 \text{ATR}_k].$$

Define:

$$S_2 = \{\Pi_2, \Pi_5, \Pi_8\}.$$

Set C — high liquidity + stable open

$$S_3 = \{\Pi_4, \Pi_6, \Pi_7\}.$$

Now define the super-set:

$$S_{\alpha, \text{short_premium}} = \{S_1, S_2, S_3\}.$$

Then:

$$\Pi(S_{\alpha, \text{short_premium}}; p_k) = \frac{1}{3} (\Pi(S_1; p_k) + \Pi(S_2; p_k) + \Pi(S_3; p_k)).$$

Concrete numeric example

Suppose at bar k :

- $\Pi(S_1; p_k) = 1$
- $\Pi(S_2; p_k) = 0$
- $\Pi(S_3; p_k) = 1$

Then:

$$\Pi(S_{\alpha, \text{short_premium}}; p_k) = \frac{1}{3} (1 + 0 + 1) = 0.667.$$

Interpretation:

Two independent logical *routes* to “short-premium-friendly” conditions are active.

This scalar can now be used as:

- an **input** to r_k (regime strength),
- a **multiplier in fuzzy sizing**,
- a **gate in entry probability**,
- or a **weight in $A_\theta(u_k)$** to adjust contractiveness.

4. HOW THIS INTEGRATES WITH CONDORNET MATH (VERY IMPORTANT)

4.1 Regime state update

$$r_k^{(\alpha)} := \Pi(S_\alpha; p_k).$$

This becomes one coordinate of r_k .

4.2 Drift modulation example

$$A_{hh}(u_k) = -\text{diag} \left[\frac{1}{1 + \lambda r_k^{(\alpha)}} \right] (\exp(\eta(u_k)) \odot (1 + \lambda r_k^{(\alpha)})).$$

- Stronger regime activation $\rightarrow$ stronger decay $\rightarrow$ tighter risk control.

4.3 Entry gating example

$$p_k^{\text{enter}} \leftarrow p_k^{\text{enter}} \cdot \Pi(S_{\alpha, \text{short_premium}}; p_k).$$

4.4 Fuzzy sizing example

Include $r_k^{(\alpha)}$ as a fuzzy input:

- IF $r_k^{(\alpha)}$ is **High** AND drawdown is **Low**
- THEN size is **Large**

5. WHY SUPER-SET ACTIVATIONS ARE POWERFUL (AND NON-OBVIOUS)

- They **compress combinatorial logic** (hundreds of inequalities) into a single scalar.
- They preserve **interpretability** (each S_a is readable).
- They avoid brittle all-or-nothing logic.
- They integrate *cleanly* with continuous neural dynamics.

This is exactly the right abstraction level between:

- symbolic logic (rules),
- and continuous control (CondorNet state evolution).

6. AUTOMATIC GENERATION OF PREDICATE SETS FROM FEATURE SCHEMAS

A “feature schema” is just metadata that tells the engine:

- what columns exist,
- what their semantic role is (price, volume, vol, microstructure, regime),
- what **canonical transforms** are allowed (returns, ratios, z-scores, ranks),
- what thresholds to use (static, percentile-based, z-score-based, ATR-scaled),
- and how to compose them into atomic predicates $\rightarrow$ sets $\rightarrow$ super-sets.

6.1 Feature schema (canonical minimal form)

A practical schema object (can be YAML/JSON/dict) looks like this:

- **columns:** mapping from semantic name to dataframe column
- **transforms:** library of derived signals
- **predicate_templates:** reusable predicate constructors
- **set_templates:** named groups of predicates (AND / OR)
- **superset_templates:** named groups of sets, aggregated into scalar activations

Example schema (conceptual):

- Price: open, high, low, close
- Volume: volume
- Volatility: atr, vol_ewma, ivr
- Microstructure: spread_ratio, te
- Momentum: macd_norm, rsi_dyn
- Risk: gap_risk_score, max_dd_60m

The generator's job is to *bind* these names to actual columns, then instantiate predicates like:

- “close-in-top-30% of bar”
- “range < 0.6 * ATR”
- “volume > 1.5 * EMA20(volume)”
- “spread_ratio < 0.001”
- “gap_risk_score < 0.4”

and then bundle them into sets and supersets.

7. PYTHON REFERENCE IMPLEMENTATION (PREDICATE → SET → SUPER-SET)

Below is a **complete, minimal reference** that you can drop into your repo (e.g., `intelligence/predicate_engine.py`). It supports:

- Atomic predicates (boolean series)
- Predicate sets with AND/OR
- Super-set activations (mean of set activations)
- Full attribution logs per bar (which atomics and sets fired)

```
1. from __future__ import annotations
2.
3. from dataclasses import dataclass
4. from typing import Callable, Dict, List, Literal, Tuple, Any
5. import numpy as np
6. import pandas as pd
7. import json
```

```
8.
9.
10. # =====
11. # Core predicate structures
12. # =====
13.
14. PredicateFn = Callable[[pd.DataFrame], pd.Series] # returns boolean Series indexed like df
15.
16. @dataclass(frozen=True)
17. class AtomicPredicate:
18.     name: str
19.     fn: PredicateFn
20.     desc: str
21.
22.
23. @dataclass(frozen=True)
24. class PredicateSet:
25.     name: str
26.     members: List[str] # atomic predicate names
27.     mode: Literal["AND", "OR"] = "AND"
28.     desc: str = ""
29.
30.
31. @dataclass(frozen=True)
32. class SuperSet:
33.     name: str
34.     members: List[str] # predicate set names
35.     agg: Literal["MEAN"] = "MEAN"
36.     desc: str = ""
37.
38.
39. # =====
40. # Feature transforms
41. # =====
42.
43. def ema(series: pd.Series, span: int) -> pd.Series:
44.     return series.ewm(span=span, adjust=False).mean()
45.
46. def log_return(close: pd.Series) -> pd.Series:
47.     return np.log(close / close.shift(1))
48.
49. def safe_div(a: pd.Series, b: pd.Series, eps: float = 1e-12) -> pd.Series:
50.     return a / (b.replace(0, np.nan).fillna(eps))
51.
52.
53. # =====
54. # Atomic predicate templates
55. # =====
56.
57. def pred_close_in_top_quantile(
58.     o: str, h: str, l: str, c: str, q: float, name: str
59. ) -> AtomicPredicate:
60.     # (C-L)/(H-L) > q
61.     def _fn(df: pd.DataFrame) -> pd.Series:
62.         rng = (df[h] - df[l]).replace(0, np.nan)
63.         rel = (df[c] - df[l]) / rng
64.         return rel > q
65.     return AtomicPredicate(
66.         name=name,
67.         fn=_fn,
68.         desc=f"(C-L)/(H-L) > {q}"
69.     )
70.
71. def pred_range_lt_atr(
72.     h: str, l: str, atr: str, mult: float, name: str
```

```
73. ) -> AtomicPredicate:
74.     def _fn(df: pd.DataFrame) -> pd.Series:
75.         return (df[h] - df[l]) < (mult * df[atr])
76.     return AtomicPredicate(
77.         name=name,
78.         fn=_fn,
79.         desc=f"(H-L) < {mult}*ATR"
80.     )
81.
82. def pred_close_gt_vwap(c: str, vwap: str, name: str) -> AtomicPredicate:
83.     def _fn(df: pd.DataFrame) -> pd.Series:
84.         return df[c] > df[vwap]
85.     return AtomicPredicate(
86.         name=name,
87.         fn=_fn,
88.         desc="C > VWAP"
89.     )
90.
91. def pred_volume_gt_ema(
92.     v: str, span: int, mult: float, name: str
93. ) -> AtomicPredicate:
94.     def _fn(df: pd.DataFrame) -> pd.Series:
95.         v_ema = ema(df[v], span=span)
96.         return df[v] > (mult * v_ema)
97.     return AtomicPredicate(
98.         name=name,
99.         fn=_fn,
100.         desc=f"V > {mult}*EMA{span}(V)"
101.     )
102.
103. def pred_spread_ratio_lt(
104.     spread_ratio: str, thresh: float, name: str
105. ) -> AtomicPredicate:
106.     def _fn(df: pd.DataFrame) -> pd.Series:
107.         return df[spread_ratio] < thresh
108.     return AtomicPredicate(
109.         name=name,
110.         fn=_fn,
111.         desc=f"spread_ratio < {thresh}"
112.     )
113.
114. def pred_gap_lt_atr(
115.     o: str, c_prev: str, atr_prev: str, mult: float, name: str
116. ) -> AtomicPredicate:
117.     # |O_k - C_{k-1}| < mult * ATR_{k-1}
118.     def _fn(df: pd.DataFrame) -> pd.Series:
119.         gap = (df[o] - df[c_prev]).abs()
120.         return gap < (mult * df[atr_prev])
121.     return AtomicPredicate(
122.         name=name,
123.         fn=_fn,
124.         desc=f"|O - prevC| < {mult}*prevATR"
125.     )
126.
127.
128. # =====
129. # Engine: evaluate atomics, sets, supersets
130. # =====
131.
132. class PredicateEngine:
133.     def __init__(
134.         self,
135.         atomics: List[AtomicPredicate],
136.         sets: List[PredicateSet],
137.         supersets: List[SuperSet],
```

```
138.     ) -> None:
139.         self.atomics = {p.name: p for p in atomics}
140.         self.sets = {s.name: s for s in sets}
141.         self.supersets = {ss.name: ss for ss in supersets}
142.
143.         # Basic sanity checks
144.         for s in sets:
145.             missing = [m for m in s.members if m not in self.atomics]
146.             if missing:
147.                 raise ValueError(f"PredicateSet '{s.name}' references unknown atomics:
{missing}")
148.
149.         for ss in supersets:
150.             missing = [m for m in ss.members if m not in self.sets]
151.             if missing:
152.                 raise ValueError(f"SuperSet '{ss.name}' references unknown sets: {missing}")
153.
154.     def eval_atomics(self, df: pd.DataFrame) -> pd.DataFrame:
155.         out = {}
156.         for name, pred in self.atomics.items():
157.             out[name] = pred.fn(df).fillna(False).astype(bool)
158.         return pd.DataFrame(out, index=df.index)
159.
160.     def eval_sets(self, atomics_df: pd.DataFrame) -> pd.DataFrame:
161.         out = {}
162.         for name, s in self.sets.items():
163.             cols = atomics_df[s.members]
164.             if s.mode == "AND":
165.                 out[name] = cols.all(axis=1)
166.             elif s.mode == "OR":
167.                 out[name] = cols.any(axis=1)
168.             else:
169.                 raise ValueError(f"Unknown set mode: {s.mode}")
170.         return pd.DataFrame(out, index=atomics_df.index)
171.
172.     def eval_supersets(self, sets_df: pd.DataFrame) -> pd.DataFrame:
173.         out = {}
174.         for name, ss in self.supersets.items():
175.             cols = sets_df[ss.members].astype(float)
176.             if ss.agg == "MEAN":
177.                 out[name] = cols.mean(axis=1)
178.             else:
179.                 raise ValueError(f"Unknown superset agg: {ss.agg}")
180.         return pd.DataFrame(out, index=sets_df.index)
181.
182.     def eval_all(self, df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
183.         atomics_df = self.eval_atomics(df)
184.         sets_df = self.eval_sets(atomics_df)
185.         supersets_df = self.eval_supersets(sets_df)
186.         return atomics_df, sets_df, supersets_df
187.
188.
189. # =====
190. # Schema-driven generation (automatic)
191. # =====
192.
193. def build_engine_from_schema(schema: Dict[str, Any]) -> PredicateEngine:
194.     cols = schema["columns"]
195.     params = schema.get("params", {})
196.
197.     atomics: List[AtomicPredicate] = []
198.
199.     # ---- Atomic predicates from templates ----
200.     atomics.append(pred_close_in_top_quantile(
201.         o=cols["open"], h=cols["high"], l=cols["low"], c=cols["close"],
```

```
202.         q=params.get("close_top_q", 0.7),
203.         name="close_in_top_30pct"
204.     ))
205.
206.     atomics.append(pred_range_lt_atr(
207.         h=cols["high"], l=cols["low"], atr=cols["atr"],
208.         mult=params.get("range_atr_mult", 0.6),
209.         name="range_lt_0p6_atr"
210.     ))
211.
212.     if "vwap" in cols:
213.         atomics.append(pred_close_gt_vwap(
214.             c=cols["close"], vwap=cols["vwap"],
215.             name="close_gt_vwap"
216.         ))
217.
218.     atomics.append(pred_volume_gt_ema(
219.         v=cols["volume"],
220.         span=params.get("vol_ema_span", 20),
221.         mult=params.get("vol_ema_mult", 1.5),
222.         name="vol_gt_1p5_ema20"
223.     ))
224.
225.     atomics.append(pred_spread_ratio_lt(
226.         spread_ratio=cols["spread_ratio"],
227.         thresh=params.get("spread_ratio_max", 0.001),
228.         name="spread_ok"
229.     ))
230.
231.     # For gap predicate we need prev close and prev atr columns.
232.     # You can store these in df precomputed; here we reference schema names.
233.     atomics.append(pred_gap_lt_atr(
234.         o=cols["open"],
235.         c_prev=cols.get("prev_close", "__prev_close__"),
236.         atr_prev=cols.get("prev_atr", "__prev_atr__"),
237.         mult=params.get("gap_atr_mult", 0.5),
238.         name="gap_ok"
239.     ))
240.
241.     # ---- Sets ----
242.     sets: List[PredicateSet] = [
243.         PredicateSet(
244.             name="bull_compress",
245.             members=["close_in_top_30pct", "range_lt_0p6_atr", "close_gt_vwap"],
246.             mode="AND",
247.             desc="Bullish dominance + compression + above VWAP"
248.         ),
249.         PredicateSet(
250.             name="exec_safe",
251.             members=["spread_ok", "gap_ok"],
252.             mode="AND",
253.             desc="Execution sanity: spreads and gaps ok"
254.         ),
255.         PredicateSet(
256.             name="participation",
257.             members=["vol_gt_1p5_ema20"],
258.             mode="AND",
259.             desc="Participation expansion"
260.         ),
261.     ]
262.
263.     # ---- Super-sets ----
264.     supersets: List[SuperSet] = [
265.         SuperSet(
266.             name="short_premium_regime",
```

```
267.         members=["bull_compress", "exec_safe", "participation"],
268.         agg="MEAN",
269.         desc="Scalar summary of multiple logical routes into short-premium regime"
270.     )
271. ]
272.
273.     return PredicateEngine(atomics=atomic, sets=sets, supersets=supersets)
274.
```

7.1 Example schema + example usage on OHLCV data

```
1. schema = {
2.     "columns": {
3.         "open": "open",
4.         "high": "high",
5.         "low": "low",
6.         "close": "close",
7.         "volume": "volume",
8.         "atr": "atr_pct",           # or "atr" if you have raw ATR
9.         "vwap": "vwap",           # optional
10.        "spread_ratio": "spread_ratio",
11.        "prev_close": "close_prev", # you can create this column
12.        "prev_atr": "atr_prev",    # you can create this column
13.    },
14.    "params": {
15.        "close_top_q": 0.7,
16.        "range_atr_mult": 0.6,
17.        "vol_ema_span": 20,
18.        "vol_ema_mult": 1.5,
19.        "spread_ratio_max": 0.001,
20.        "gap_atr_mult": 0.5,
21.    }
22. }
23.
24. engine = build_engine_from_schema(schema)
25.
26. # Precompute required "prev" columns if not present:
27. df = df.copy()
28. df["close_prev"] = df["close"].shift(1)
29. df["atr_prev"] = df["atr_pct"].shift(1)
30.
31. atomic_df, sets_df, supersets_df = engine.eval_all(df)
32.
33. # Example: attach the scalar superset activation as a regime coordinate
34. df["r_short_premium"] = supersets_df["short_premium_regime"]
35.
```

8. BACK-ATTRIBUTION: WHICH PREDICATE SETS CONTRIBUTED TO A TRADE (AUDIT LOGS)

You want to be able to say, for a given trade decision at bar k :

- which atomic predicates fired,
- which sets fired,
- which supersets were high,
- and the final scalar value used by the model (e.g., appended into r_k or used as a gate).

8.1 Audit log record format (JSONL)

A single bar-level log record:

- includes IDs/timestamps,
- includes the decision output (enter/exit/size),
- includes predicate activations,
- includes set activations,
- includes superset scalar values,

```
1. includes contribution lists (what mattered).
2. def build_audit_record(
3.     k: int,
4.     ts: Any,
5.     decision: Dict[str, Any],
6.     atomics_row: pd.Series,
7.     sets_row: pd.Series,
8.     supersets_row: pd.Series,
9.     top_n: int = 10,
10. ) -> Dict[str, Any]:
11.     # Atomics: list which are true
12.     true_atomics = [name for name, val in atomics_row.items() if bool(val)]
13.     true_sets = [name for name, val in sets_row.items() if bool(val)]
14.
15.     # Supersets: sort by scalar value
16.     sup_sorted = sorted(
17.         [(name, float(val)) for name, val in supersets_row.items()],
18.         key=lambda x: x[1],
19.         reverse=True
20.     )
21.
22.     # Contribution heuristic:
23.     # - a superset contributes if it is non-trivial (>0)
24.     # - within it, contributing sets are those true
25.     # - within contributing sets, contributing atomics are those true
26.     contributions = []
27.     for sup_name, sup_val in sup_sorted:
28.         if sup_val <= 0.0:
29.             continue
30.         # Find member sets for this superset (requires access to schema/engine externally if you
31.         # want exact members)
32.         contributions.append({
33.             "superset": sup_name,
34.             "value": sup_val,
35.             "active_sets": true_sets,
36.             "active_atomics": true_atomics,
37.         })
38.     return {
39.         "k": int(k),
40.         "timestamp": str(ts),
41.         "decision": decision, # e.g. {"enter":1,"exit":0,"size":3}
42.         "predicates": {
43.             "atomics_true": true_atomics[:top_n],
44.             "sets_true": true_sets[:top_n],
45.             "supersets_top": sup_sorted[:top_n],
46.         },
47.         "contributions": contributions[:top_n],
48.     }
49.
```

8.2 Write JSONL audit logs alongside trade decisions

```
1. def append_audit_jsonl(path: str, record: Dict[str, Any]) -> None:
2.     with open(path, "a", encoding="utf-8") as f:
3.         f.write(json.dumps(record, ensure_ascii=False) + "\n")
4.
5.
6. # Example integration loop:
7. audit_path = "decision_trace.jsonl"
8. for k in range(len(df)):
9.     # Pretend these are computed by your model/arbitrator
10.    decision = {
11.        "enter": int(df.loc[df.index[k], "enter_signal"]),
12.        "exit": int(df.loc[df.index[k], "exit_signal"]),
13.        "size": int(df.loc[df.index[k], "contracts"]),
14.    }
15.
16.    record = build_audit_record(
17.        k=k,
18.        ts=df.index[k],
19.        decision=decision,
20.        atomics_row=atomics_df.iloc[k],
21.        sets_row=sets_df.iloc[k],
22.        supersets_row=supersets_df.iloc[k],
23.        top_n=20
24.    )
25.    append_audit_jsonl(audit_path, record)
26.
```

8.3 Back-attribution at trade-level (entry → exit window)

To explain *a trade*, aggregate logs over its active bars:

- take the mean/max of superset activations over the holding window,
- count which sets fired most,
- count which atomic predicates were persistently true,
- output a compact “why we entered / why we exited” summary.

```
1. def summarize_trade_window(
2.     records: List[Dict[str, Any]],
3.     superset_key: str = "short_premium_regime",
4. ) -> Dict[str, Any]:
5.     # Extract superset values
6.     vals = []
7.     set_counts: Dict[str, int] = {}
8.     atomic_counts: Dict[str, int] = {}
9.
10.    for r in records:
11.        # supersets_top is list of (name, value)
12.        sup_map = dict(r["predicates"]["supersets_top"])
13.        if superset_key in sup_map:
14.            vals.append(float(sup_map[superset_key]))
15.
16.        for s in r["predicates"]["sets_true"]:
17.            set_counts[s] = set_counts.get(s, 0) + 1
18.        for a in r["predicates"]["atomics_true"]:
19.            atomic_counts[a] = atomic_counts.get(a, 0) + 1
20.
21.    # Rank by frequency
22.    top_sets = sorted(set_counts.items(), key=lambda x: x[1], reverse=True)[:10]
23.    top_atomics = sorted(atomic_counts.items(), key=lambda x: x[1], reverse=True)[:15]
```

```
24.  
25.     return {  
26.         "superset": superset_key,  
27.         "superset_mean": float(np.mean(vals)) if vals else 0.0,  
28.         "superset_max": float(np.max(vals)) if vals else 0.0,  
29.         "top_sets_by_frequency": top_sets,  
30.         "top_atomics_by_frequency": top_atomics,  
31.         "bars_in_trade": len(records),  
32.     }  
33.
```

9. WHERE THIS PLUGS INTO OUR CONDORNET MATH (THE EXACT BRIDGE)

If you define a superset activation:

$$r_k^{(\alpha)} := \Pi(S_\alpha; p_k) \in [0,1],$$

then in code we literally do:

- compute supersets scalar series,
- append as regime coordinates:

```
1. df["r_short_premium"] = supersets_df["short_premium_regime"]  
2. # Then r_k = [ ... , r_short_premium, ... ]  
3.
```

And then r_k enters the CondorNet state $x_k = [h_k; v_k; m_k; r_k]$ **and/or** the forcing term $D(\cdot)$, and/or the operator $A_\theta(u_k)$.